# Sentinel — Detailed Project Report

Comprehensive explanation of the project, architecture, data, training, inference, and deep AI/ML description.

## 1. Project Overview

Sentinel is a full-stack credit card fraud detection system combining a React frontend, a Nitro Node backend that delegates ML inference to Python, and a Random Forest classifier trained on the creditcard.csv dataset.

## 2. Architecture

Frontend (React): transaction UI, bulk upload, predictions display; Backend (Node/Nitro): API endpoints, spawns Python inference worker; ML (Python): training and prediction scripts (train.py, predict.py, predict_batch.py).

## 3. Data & Features

The dataset is the standard creditcard.csv containing Time, V1..V28 (PCA features), Amount and Class (0/1 fraud label). Feature order used for training & inference: [Time, V1..V28, Amount].

## 4. Training Pipeline (train.py)

Key steps implemented in train.py:

- Load dataset and separate X (features) and y (target).
- Scale all features using StandardScaler to normalize distributions.
- Perform stratified train/test split to preserve class imbalance ratio.
- Train RandomForestClassifier with n_estimators=100, max_depth=12, class_weight='balanced'.
- Evaluate with confusion matrix, precision, recall, F1, ROC-AUC.
- Persist model (model.joblib), scaler (scaler.joblib) and metrics (model_metrics.json).

## 5. Model Evaluation & Metrics

Recorded metrics:

| Metric | Value |
|---|---|
| accuracy | 99.91 |
| precision | 71.68 |
| recall | 82.65 |
| f1_score | 76.78 |

| | |
|---|---|
| roc_auc | 97.9 |

Confusion matrix (from metrics file):

| | Predicted Genuine | Predicted Fraud |
|---|---|---|
| Actual Genuine | 56832 | 32 |
| Actual Fraud | 17 | 81 |

## 6. Inference Pipeline (predict.py & predict_batch.py)

predict.py accepts a JSON payload with keys: amount, time, features (V1..V28). It converts time to seconds, constructs the feature vector [time_seconds, V1..V28, amount], scales using saved StandardScaler, and runs model.predict_proba to return fraud probability. For batch operations, predict_batch.py reads a JSON array from stdin and returns an array of {fraud, prob}.

## 7. Frontend Integration

The React app sends JSON to /api/predict and /api/predict-batch. The Node server spawns the Python worker and proxies the JSON result back to the UI, which displays a gauge, risk level, recommended action, and a transaction summary.

## 8. Deep AI/ML Explanation

Data preprocessing and scaling:

StandardScaler centers each feature to mean 0 and unit variance. This matters because the Random Forest uses thresholds and split criteria that can be affected by feature scale; although tree models are less sensitive to scaling than linear models, scaling the PCA components and Amount/Time ensures consistent numerical stability and preserves relative contributions across features.

Why PCA-like V1..V28 features are present:

The original dataset provides anonymized PCA transformed features. These components capture orthogonal directions of variance in the transaction feature space. The model uses these transformed signals rather than raw cardholder attributes to learn discriminative patterns while protecting privacy.

Model selection and tradeoffs:

Random Forests combine many decision trees trained on bootstrap samples. Each tree produces a probability vote; averaging yields robust probabilities. Trees capture nonlinearities and interactions between PCA features and time/amount. The main tradeoffs: interpretability (trees less interpretable than linear models but more than deep nets), computational cost (inference cost scales with n_estimators), and overfitting (controlled with max_depth, min_samples_leaf).

Handling class imbalance:

Because fraud is rare, we use stratified splits and class_weight='balanced'. This tells the tree learner to weight minority class errors more heavily, improving recall at some precision cost. Additional techniques (not implemented here) include: SMOTE/oversampling, anomaly-detection hybrid approaches, cost-sensitive learning, and ensemble stacking with calibrated probabilities.

Probability calibration & thresholding:

The model returns probabilities. The system threshold is currently 0.5; recommended production steps are to calibrate probabilities (Platt scaling or isotonic regression) and select an operating point balancing false positives vs detection rate based on expected fraud loss and investigation costs.

Explainability & monitoring:

For analyst workflows, add SHAP value explanation to show feature contributions per transaction; add drift detection on feature distributions; and schedule periodic retraining with recent labeled data to handle behavior drift.

## 9. Run & Verification Checklist

```
Install frontend dependencies: cd clarity-fraud-watch-main && npm install
Install Python packages: python -m pip install numpy pandas scikit-learn joblib
reportlab
Start frontend: cd clarity-fraud-watch-main && npm run dev
Run training (optional): python train.py
Test single prediction: python predict.py sample_transaction.json
Test batch prediction: python predict_batch.py < sample_batch.json
```

## 10. Recommendations for production hardening

- Add probability calibration and select operating point using cost-sensitive metrics.

- Integrate SHAP explanations into the UI to aid analysts.

- Add continuous evaluation, drift detection, and an automatic retraining pipeline.

- Consider using a gradient boosting framework (LightGBM/XGBoost/CatBoost) for improved performance and faster training.

- Add secure logging, rate-limiting, and an approval workflow for blocked transactions.

Generated by the Sentinel project assistant. Contact the project owner for dataset access and retraining instructions.